

Robótica Computacional

Grado en Ingeniería Informática (cuarto curso)

Apuntes de Python

Estilo

Guía de estilo estándar para Python: <http://legacy.python.org/dev/peps/pep-0008/>

Recomendaciones:

- Indentación a 4 espacios
- Organizar las líneas para que no excedan los 79 caracteres.
- Usar líneas en blanco para separar funciones, clases y bloques de código.
- Añadir comentarios concisos en la medida de lo posible.
- Usar docstrings.
- Poner espacios rodeando operadores y después de comas, salvo dentro de delimitadores: `a = f(1, 2) + g(3, 4)`.
- Utilizar una convención de nombres homogénea: CamelCase para clases y `lower_case_with_underscores` para funciones y métodos.

Intérprete

```
$ python
Python 3.6.9 (default,(default, Mar 10 2023, 16:46:00)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
>>> the_world_is_flat = 1
>>> if the_world_is_flat:
...     print ("Be careful not to fall off!")
...
Be careful not to fall off!
```

```
>>>
KeyboardInterrupt
>>> q
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'q' is not defined
>>> exit
Use exit() or Ctrl-D (i.e. EOF) to exit
```

Usando Python como calculadora

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5.0*6) / 4
5.0
>>> 8 / 5.0
1.6
```

```
>>> 17 / 3 # int / int -> int
5
>>> 17 / 3.0 # int / float -> float
5.666666666666667
>>> 17 // 3.0 # explicit floor division discards the fractional part
5.0
>>> 17 % 3 # the % operator returns the remainder of the division
2
>>> 5 * 3 + 2 # result * divisor + remainder
17
```

```
>>> 5 ** 2 # 5 squared
25
>>> 2 ** 7 # 2 to the power of 7
128
```

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900
```

```
>>> n # try to access an undefined variable
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

```
>>> 3 * 3.75 / 1.5
7.5
>>> 7.0 / 2
3.5
```

```
>>> tax = 12.5 / 100
>>> price = 100.50
>>> price * tax
12.5625
>>> price + _
113.0625
>>> round(_, 2)
113.06
```

Strings

```
>>> 'spam eggs' # single quotes
'spam eggs'
>>> 'doesn\'t' # use \' to escape the single quote...
"doesn't"
>>> "doesn't" # ...or use double quotes instead
"doesn't"
>>> '"Yes," he said.'
'"Yes," he said.'
>>> "\"Yes,\" he said."
'"Yes," he said.'
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
```

Combinando strings

```
>>> # 3 times 'un', followed by 'ium'
>>> 3 * 'un' + 'ium'
'unununium'
```

```
>>> 'Py' 'thon'
'Python'
```

```
>>> prefix = 'Py'
>>> prefix 'thon' # can't concatenate a variable and a string literal
...
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
...
SyntaxError: invalid syntax
```

```
>>> prefix + 'thon'
'Python'
```

```
>>> text = ('Put several strings within parentheses '
            'to have them joined together.')
>>> text
'Put several strings within parentheses to have them joined together.'
```

Indexando Strings

```
>>> word = 'Python'
>>> word[0]  # character in position 0
'P'
>>> word[5]  # character in position 5
'n'
>>> word[-1] # last character
'n'
>>> word[-2] # second-last character
'o'
>>> word[-6]
'P'
```

+	-	-	+	-	-	+	-	-	+	-	-	+
	P		y		t		h		o		n	
+	-	-	+	-	-	+	-	-	+	-	-	+
0	1	2	3	4	5	6						
-6	-5	-4	-3	-2	-1							

Slicing

```
>>> word[:]  
'Python'  
>>> word[0:2]  # characters from position 0 (included) to 2 (excluded)  
'Py'  
>>> word[2:5]  # characters from position 2 (included) to 5 (excluded)  
'tho'  
>>> word[:2] + word[2:]  
'Python'  
>>> word[:4] + word[4:]  
'Python'  
>>> word[:2]  # character from the beginning to position 2 (excluded)  
'Py'  
>>> word[4:]  # characters from position 4 (included) to the end  
'on'  
>>> word[-2:] # characters from the second-last (included) to the end  
'on'  
>>> word[0] = 'J'  
...  
TypeError: 'str' object does not support item assignment  
>>> word[2:] = 'py'  
...  
TypeError: 'str' object does not support item assignment  
>>> 'J' + word[1:]  
'Jython'  
>>> word[:2] + 'py'  
'Pypy'  
>>> s = 'supercalifragilisticexpialidocious'  
>>> len(s)  
34
```

Control de flujo - if

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print( "Negative")
... elif x == 0:
...     print("Zero")
... elif x == 1:
...     print("Uno")
... else:
...     print("More")
...
More
```

Control de flujo - for

```
>>> words= ['cat','window','boat']
>>> for w in words:
...     print(w,len(w))
...
cat 3
window 6
boat 4

for w in words[:]:
...     if len(w) > 5:
...         words.insert(0,w)
...
>>> words
['window', 'cat', 'window', 'boat']
```

Control de flujo - for (range)

```
>>> for i in range(len(words)):
...     print(i, words[i])
...
0 window
1 cat
2 window
3 boat
```

Control de flujo – break y else

```
>>> for n in range(2,10):  
...     for x in range(2,n):  
...         if n % x == 0:  
...             print(n, ' equals ', x, '*', n/x)  
...             break  
...     else:  
...         print(n, ' is a prime number ')  
...  
2 is a prime number  
3 is a prime number  
4 equals 2 * 2.0  
5 is a prime number  
6 equals 2 * 3.0  
7 is a prime number  
8 equals 2 * 4.0  
9 equals 3 * 3.0
```

Control de flujo - continue

```
>>> for num in range(2,10):  
...     if num % 2 == 0:  
...         print('Found a even number', num)  
...         continue  
...     print ('Found a number', num)  
...  
Found a even number 2  
Found a number 3  
Found a even number 4  
Found a number 5  
Found a even number 6  
Found a number 7  
Found a even number 8  
Found a number 9
```

Definiendo funciones

```
>>> def fib(n): # write a fibonacci series
```

```
...     a,b = 0,1
```

```
...     while a < n:
```

```
...         print (a)
```

```
...         a,b = b, a+b
```

```
...
```

```
>>> fib(2000)
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
5
```

```
8
```

```
13
```

```
21
```

```
34
```

```
55
```

```
89
```

```
144
```

```
233
```

```
377
```

```
610
```

```
987
```

```
1597
```

```
>>> fib(0)
```

```
>>> print(fib(0))
```

```
None
```

```
>>> fib
```

```
<function fib at 0x7f2947dec950>
```

```
>>> f = fib
```

```
>>> f(100)
```

```
0
```

```
1
```

```
1
```

```
2
```

```
3
```

```
5
```

```
8
```

```
13
```

```
21
```

```
34
```

```
55
```

```
89
```

Definiendo funciones

```
>>> def fib(n): # return Fibonacci series up to n
...     """Return a list containing the Fibonacci series up to n."""
...     result = []
...     a, b = 0, 1
...     while a < n:
...         result.append(a)    # see below
...         a, b = b, a+b
...     return result
...
>>> f100 = fib(100)    # call it
>>> f100               # write the result
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```


Funciones – Argumentos por defecto

```
>>> def ask_ok(prompt, retries=4, complaint='Yes or no, please!'):
...     while True:
...         ok = input(prompt)
...         if ok in ('y', 'ye', 'yes'):
...             return True
...         if ok in ('n', 'no', 'nop', 'nope'):
...             return False
...         retries = retries - 1
...         if retries < 0:
...             raise IOError('refusenik user')
...         print (complaint)
>>> ask_ok('Do you really want to quit?')
Do you really want to quit?
Yes or no, please!
Do you really want to quit?
Yes or no, please!
Do you really want to quit?
Yes or no, please!
Do you really want to quit?
Yes or no, please!
Do you really want to quit?
Yes or no, please!
Do you really want to quit?
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 10, in ask_ok
IOError: refusenik user
```

Funciones – Argumentos por defecto (evaluación)

```
i = 5
>>> def f(arg=i):
...     print(arg)
...
>>> f()
5

>>> def f(a, L=[]):
...     L.append(a)
...     return L
...
>>> print(f(1))
[1]
>>> print(f(2))
[1, 2]
>>> print(f(3))
[1, 2, 3]
```

Funciones - keywords

```
def parrot(voltage, state='a stiff', action='vroom', type='Norwegian Blue'):  
    print("-- This parrot wouldn't", action)  
    print("if you put", voltage, "volts through it.")  
    print("-- Lovely plumage, the", type)  
    print("-- It's", state, "!")
```

```
parrot(1000) # 1 positional argument  
parrot(voltage=1000) # 1 keyword argument  
parrot(voltage=1000000, action='VOOOOOM') # 2 keyword arguments  
parrot(action='VOOOOOM', voltage=1000000) # 2 keyword arguments  
parrot('a million', 'bereft of life', 'jump') # 3 positional arguments  
parrot('a thousand', state='pushing up the daisies') # 1 positional, 1 keyword
```

```
parrot() # required argument missing  
parrot(voltage=5.0, 'dead') # non-keyword argument after a keyword argument  
parrot(110, voltage=220) # duplicate value for the same argument  
parrot(actor='John Cleese') # unknown keyword argument
```

Funciones – Desempaquetado de argumentos y funciones lambda

```
>>> args = [3, 6]
>>> range(*args)           # call with arguments unpacked from a list
```

```
>>> def make_incrementor(n):
...     return lambda x: x + n
...
>>> f = make_incrementor(42)
>>> f(0)
42
>>> f(1)
43
```

```
>>> pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
>>> pairs.sort(key=lambda pair: pair[1])
>>> pairs
[(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]
```

Funciones - docstrings

```
>>> def my_function():  
...     """Do nothing, but document it  
...  
...     No really, it doesnt do anything  
...     """  
...     pass  
...  
  
>>> print(my_function.__doc__)  
Do nothing, but document it  
  
    No really, it doesnt do anything
```

Estructuras de datos - Listas

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares
[1, 4, 9, 16, 25]
>>> squares[0] # indexing returns the item
1
>>> squares[-1]
25
>>> squares[-3:] # slicing returns a new list
[9, 16, 25]
>>> squares[:]
[1, 4, 9, 16, 25]
>>> squares + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>> cubes = [1, 8, 27, 65, 125] # something's wrong here
>>> 4 ** 3 # the cube of 4 is 64, not 65!
64
>>> cubes[3] = 64 # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
>>> cubes.append(216) # add the cube of 6
>>> cubes.append(7 ** 3) # and the cube of 7
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

Estructuras de datos - Listas

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> # replace some values
>>> letters[2:5] = ['C', 'D', 'E']
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> # now remove them
>>> letters[2:5] = []
>>> letters
['a', 'b', 'f', 'g']
>>> # clear the list by replacing all the elements with an empty list
>>> letters[:] = []
>>> letters
[]
>>> letters = ['a', 'b', 'c', 'd']
>>> len(letters)
4
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

Estructuras de datos - Listas

```
>>> a = [66.25, 333, 333, 1, 1234.5]
>>> print(a.count(333), a.count(66.25), a.count('x'))
2 1 0
>>> a.insert(2, -1)
>>> a.append(333)
>>> a
[66.25, 333, -1, 333, 1, 1234.5, 333]
>>> a.index(333)
1
>>> a.remove(333)
>>> a
[66.25, -1, 333, 1, 1234.5, 333]
>>> a.reverse()
>>> a
[333, 1234.5, 1, 333, -1, 66.25]
>>> a.sort()
>>> a
[-1, 1, 66.25, 333, 333, 1234.5]
>>> a.pop()
1234.5
>>> a
[-1, 1, 66.25, 333, 333]
```


Estructuras de datos - Listas

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")           # Terry arrives
>>> queue.append("Graham")         # Graham arrives
>>> queue.popleft()                # The first to arrive now leaves
'Eric'
>>> queue.popleft()                # The second to arrive now leaves
'John'
>>> queue                          # Remaining queue in order of arrival
deque(['Michael', 'Terry', 'Graham'])
```

Estructuras de datos - Listas

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
>>> del a
>>> a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
```

Estructuras de datos – Listas (filter, map y reduce)

```
>>> def f(x): return x % 3 == 0 or x % 5 == 0
...
>>> filter(f, range(2, 25))
[3, 5, 6, 9, 10, 12, 15, 18, 20, 21, 24]
```

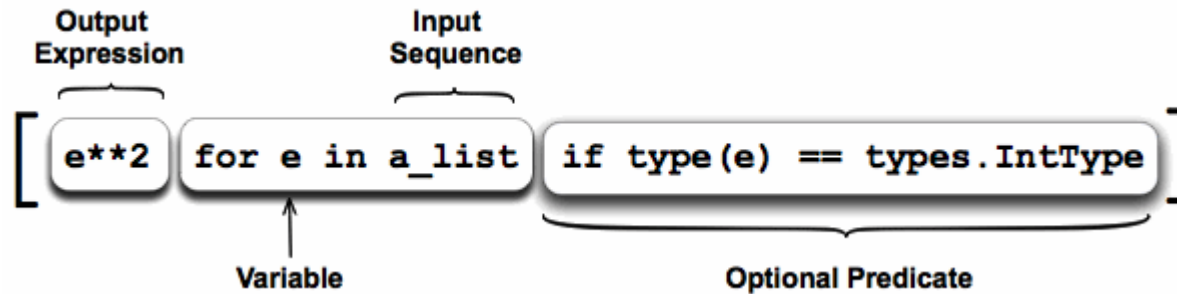
```
>>> def cube(x): return x*x*x
...
>>> map(cube, range(1, 11))
[1, 8, 27, 64, 125, 216, 343, 512, 729, 1000]
```

```
>>> seq = range(8)
>>> def add(x, y): return x+y
...
>>> map(add, seq, seq)
[0, 2, 4, 6, 8, 10, 12, 14]
```

```
>>> def add(x,y): return x+y
...
>>> reduce(add, range(1, 11))
55
```

```
>>> def sum(seq):
...     def add(x,y): return x+y
...     return reduce(add, seq, 0)
...
>>> sum(range(1, 11))
55
>>> sum([])
0
```

Estructuras de datos - List Comprehensions



```
>>> vec = [-4, -2, 0, 2, 4]
>>> # create a new list with the values doubled
>>> [x*2 for x in vec]
[-8, -4, 0, 4, 8]
>>> # filter the list to exclude negative numbers
>>> [x for x in vec if x >= 0]
[0, 2, 4]
>>> # apply a function to all the elements
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> # call a method on each element
>>> freshfruit = [' banana', ' loganberry ', 'passion fruit ']
>>> [weapon.strip() for weapon in freshfruit]
['banana', 'loganberry', 'passion fruit']
```

```
>>> squares = []
>>> for x in range(10):
...     squares.append(x**2)
...
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Estructuras de datos - List Comprehensions

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
>>> combs = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
...             combs.append((x, y))
...
>>> combs
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Estructuras de datos - List Comprehensions (anidadas)

```
>>> matrix = [  
...     [1, 2, 3, 4],  
...     [5, 6, 7, 8],  
...     [9, 10, 11, 12],  
... ]
```

```
>>> [[row[i] for row in matrix] for i in range(4)]  
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

```
>>> transposed = []  
>>> for i in range(4):  
...     transposed.append([row[i] for row in matrix])  
...  
>>> transposed  
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Estructuras de datos - Tuplas

```
>>> t = 12345, 54321, 'hello!'
>>> x, y, z = t
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> # Tuples may be nested:
... u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
>>> # Tuples are immutable:
... t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # but they can contain mutable objects:
... v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
>>> empty = ()
>>> singleton = 'hello',    # <-- note trailing comma
>>> len(empty)
0
>>> len(singleton)
1
>>> singleton
('hello',)
```

Estructuras de datos - Conjuntos

```
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> fruit = set(basket)           # create a set without duplicates
>>> fruit
set(['orange', 'pear', 'apple', 'banana'])
>>> 'orange' in fruit             # fast membership testing
True
>>> 'crabgrass' in fruit
False

>>> # Demonstrate set operations on unique letters from two words
...
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                               # unique letters in a
set(['a', 'r', 'b', 'c', 'd'])
>>> a - b                           # letters in a but not in b
set(['r', 'd', 'b'])
>>> a | b                           # letters in either a or b
set(['a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'])
>>> a & b                           # letters in both a and b
set(['a', 'c'])
>>> a ^ b                           # letters in a or b but not both
set(['r', 'd', 'b', 'm', 'z', 'l'])
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
set(['r', 'd'])
```


Estructuras de datos - Diccionarios

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> tel['jack']
4098
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'guido': 4127, 'irv': 4127, 'jack': 4098}
>>> tel.keys()
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'jack': 4098, 'guido': 4127}
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'jack': 4098, 'guido': 4127}
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.iteritems():
...     print k, v
...
gallahad the pure
robin the brave
```

Módulos

```
# Fibonacci numbers module

def fib(n):    # write Fibonacci series up to n
    a, b = 0, 1
    while b < n:
        print b,
        a, b = b, a+b

def fib2(n): # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
```

```
>>> import fibo
```

```
>>> fibo.fib(1000)
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

```
>>> fib = fibo.fib
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Módulos

```
>>> from fibo import fib, fib2
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> from fibo import *
>>> fib(500)
1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
python fibo.py <arguments>
```

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

```
$ python fibo.py 50
1 1 2 3 5 8 13 21 34
```

Python - NumPy

NumPy es un paquete de programación científica para Python.

Contiene:

- manejo de matrices de N dimensiones,
- funciones matemáticas,
- herramientas de integración con C/C++ y Fortran,
- álgebra lineal, transformadas de Fourier y aleatorización.

Guía de usuario: <http://docs.scipy.org/doc/numpy/reference>

Uso:

```
>>> import numpy as np
>>> np.sum([1,2,3,4,5])
15
```

Python - NumPy - Arrays

Creación y acceso a los elementos:

```
>>> x = np.array([[1, 2, 3], [4, 5, 6]])
>>> x
array([[1, 2, 3],
       [4, 5, 6]])
>>> x[1]      # Fila 1
array([4, 5, 6])
>>> x[1, 2]   # Fila 1, columna 2
6
>>> x[1, 2] = 9
>>> x
array([[1, 2, 3],
       [4, 5, 9]])
```

Conversión a lista:

```
>>> x.tolist()
[[1, 2, 3], [4, 5, 6]]
```

Matriz identidad:

```
>>> np.identity(4)
array([[ 1.,  0.,  0.,  0.],
       [ 0.,  1.,  0.,  0.],
       [ 0.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  1.]])
```

Python - NumPy - Arrays (2)

Mínimos, máximos, sumatorios y productos:

```
>>> x = [[1, 3, 5], [6, 4, 2]] # Lista
>>> np.min(x)
1
>>> np.max(x)
6
>>> np.sum(x)
21
>>> np.prod(x)
720
```

```
>>> x = np.array([[1, 3, 5], [6, 4, 2]]) # Array
>>> x.min()
1
>>> x.max()
6
>>> x.sum()
21
>>> x.prod()
720
```

Transpuesta de una matriz:

```
>>> np.array([[1, 2, 3], [4, 5, 6]])
>>> x
array([[1, 2, 3],
       [4, 5, 6]])
>>> x.T
array([[1, 4],
       [2, 5],
       [3, 6]])
```

Módulo de un vector:

```
>>> np.linalg.norm([4, 0, 3]) # Válido tanto para listas como para arrays
5.0
```

Python - NumPy - Arrays (3)

Sumas y restas de vectores:

```
>>> x = [1, 2, 3] # Listas
>>> y = [4, 5, 6]
>>> np.add(x, y)
array([5, 7, 9])
>>> np.subtract(x, y)
array([-3, -3, -3])
```

```
>>> x = np.array([1, 2, 3]) # Arrays
>>> y = np.array([4, 5, 6])
>>> x + y
array([5, 7, 9])
>>> x - y
array([-3, -3, -3])
```

Producto escalar:

```
>>> np.dot(x, y) # Tanto para listas como para arrays
32
```

Producto matricial:

```
>>> x = [[1, 2, 3], [4, 5, 6]]
>>> y = [[7, 6, 5, 4], [8, 4, 2, 1], [9, 7, 5, 3]]
>>> np.dot(x, y) # Tamaños compatibles: (2x3) x (3x4) -> (2x4)
array([[ 50,  35,  24,  15],
       [122,  86,  60,  39]])
>>> np.dot(y, x) # Tamaños incompatibles: (3x4) x (2x3) -> no se pueden multiplicar
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: objects are not aligned
```

Ejercicio

1. Resolver $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$ en Python

```
x = [[1,2,3],[4,5,6]]  
y = [[1,2],[3,4],[5,6]]  
...  
print ...
```

2. Resolver $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$ usando NumPy

```
import numpy as np  
x = [[1,2,3],[4,5,6]]  
y = [[1,2],[3,4],[5,6]]  
print ...
```